perplexity

Consider the attached image. what would be a technique I could use in React.js where I could make the bat fly around the tower and/or have the window light flicker or increase in intensity.

Use **layered animation**: keep the artwork as a static background, then place independently animated elements over it. For this image, that means a transparent bat layer following a flight path and a warm, translucent light layer aligned with the window.

React can render the scene; CSS and SVG can handle the animation without updating React state on every frame. CSS motion paths let you move an element along a custom curve, while SVG blur filters can create the window's soft glow.[1] [2]

How I'd approach this particular image

There are two different levels of effect:

Effect	Technique	Image preparation
Make the window flicker or brighten	Overlay a window-shaped amber highlight and blurred halo; animate their opacity	None required for an initial version
Add flying bats	Animate a transparent bat SVG or PNG over the image	None, if you're adding new bats
Animate one of the existing bats	Remove it from the background, then recreate it as a separate animated layer	Requires retouching the image
Make a bat actually pass behind the tower	Place the bat between a background layer and a transparent tower cutout	Requires a tower cutout or carefully traced occlusion mask

The important distinction: moving a bat across the image is straightforward; making it appear to circle the tower convincingly requires hiding it when it passes behind the tower.

React example: flickering window and flying bat

An SVG overlay is a useful fit here because its `viewBox` gives the animation the same coordinate system as the image. That makes the window highlight and flight path scale together with the artwork.

This example uses approximate coordinates for your 1211 × 1080 image. The window shape will need a little adjustment to align precisely.

```
import { useId } from "react";
import "./TowerScene.css";

export default function TowerScene() {
  const glowId = `window-glow-${useId().replace(/:/g, "")}`;

  return (
    <div className="tower-scene">
      <img
        className="tower-scene__image"
        src="/images/tower.jpg"
        alt="A moonlit stone tower on a rocky hill"
      />

      <svg
        className="tower-scene__effects"
        viewBox="0 0 1211 1080"
        aria-hidden="true"
      >
        <defs>
          <filter
            id={glowId}
            x="-100%"
            y="-100%"
            width="300%"
            height="300%"
          >
            <feGaussianBlur stdDeviation="15" />
          </filter>
        </defs>

        {/* Soft light spilling around the window */}
        <ellipse
          className="window-halo"
          cx="558"
          cy="415"
          rx="32"
          ry="58"
          fill="#ffb84d"
          filter={`url(#${glowId})`}
        />

        {/* Approximate window interior; refine to match the artwork */}
        <path
          className="window-light"
          d="
            M 537 455
            L 537 397
            Q 538 380 555 366
            Q 574 381 576 397
            L 576 455
            Z
          "
          fill="#ffd374"
        />
```

```
        {/* Motion belongs to the outer group; wingbeat to the inner one */}
        <g className="flying-bat">
          <g className="bat-wings">
            <path
              fill="#17130f"
              d="
                M 0 0
                C -9 -13 -23 -16 -38 -13
                L -27 -2
                L -19 -5
                L -12 5
                L -5 2
                L 0 10
                L 5 2
                L 12 5
                L 19 -5
                L 27 -2
                L 38 -13
                C 23 -16 9 -13 0 0
                Z
              "
            />
          </g>
        </g>
      </svg>
    </div>
  );
}
```

```
.tower-scene {
  position: relative;
  width: min(100%, 1000px);
  isolation: isolate;
}

.tower-scene__image {
  display: block;
  width: 100%;
  height: auto;
}

.tower-scene__effects {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  pointer-events: none;
}

/* Brighten the window without completely hiding its existing detail. */
.window-light {
  mix-blend-mode: screen;
  animation: candle-flicker 3.7s linear infinite;
}
```

```
.window-halo {
  mix-blend-mode: screen;
  animation: candle-flicker 3.7s linear infinite;
}

@keyframes candle-flicker {
  0%, 100% { opacity: 0.18; }
  8%       { opacity: 0.32; }
  11%      { opacity: 0.12; }
  16%      { opacity: 0.38; }
  23%      { opacity: 0.23; }
  47%      { opacity: 0.30; }
  49%      { opacity: 0.10; }
  53%      { opacity: 0.35; }
  72%      { opacity: 0.21; }
  85%      { opacity: 0.33; }
}

/* Path coordinates use the SVG's coordinate system. */
.flying-bat {
  offset-path: path(
    "M 820 460
     C 1050 310 1060 650 820 680
     C 650 690 680 530 820 460 Z"
  );
  offset-rotate: 0deg;
  animation: bat-flight 12s linear infinite;
}

@keyframes bat-flight {
  from { offset-distance: 0%; }
  to   { offset-distance: 100%; }
}

/* A simple stylized flap; a sprite looks more natural. */
.bat-wings {
  animation: wingbeat 180ms ease-in-out infinite alternate;
}

@keyframes wingbeat {
  from { transform: scaleY(1); }
  to   { transform: scaleY(0.35); }
}

@media (prefers-reduced-motion: reduce) {
  .window-light,
  .window-halo,
  .flying-bat,
  .bat-wings {
    animation: none;
  }

  .window-light,
  .window-halo {
    opacity: 0.2;
```

```
    }
  }
```

The bat travels by animating `offset-distance` from `0%` to `100%` along its `offset-path`; keeping `offset-rotate` fixed prevents it from turning upside down as it follows the loop. The window halo uses `feGaussianBlur`; the expanded filter bounds keep the blurred light from being clipped.[2] [3] [4]

This sample adds a bat flying in the sky to the right of the tower—it does not yet implement behind-the-tower occlusion.

Making it look convincing

For your artwork, I would prioritize these refinements:

- Window detail: trace the individual glass panes instead of covering the entire opening. That preserves the dark stone frame and mullions.
- Actual flicker versus brightening: the current overlay can add light, but it cannot extinguish the illumination already baked into the image. For a window that goes nearly dark, prepare a darker base image first.
- Bat appearance: use a transparent bat asset that matches the illustration's rough silhouette and muted colors. A few wing-animation frames will look better than the simple squash animation above.
- Tower occlusion: use this layer order: background → flying bat → transparent tower cutout → window glow. The tower cutout naturally hides the bat as it crosses behind it.
- Depth: make the bat smaller during the back half of its orbit and larger during the front half.

For a first pass, I'd start with the window overlay and one added bat. That gives you an animated scene without needing to separate the whole illustration into layers.

⁂

1. https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Motion_path
2. https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/feGaussianBlur
3. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/offset-path
4. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/offset-distance
5. https://github.com/mdn/content/blob/main/files/en-us/web/api/svgfegaussianblurelement/x/index.md?plain=1
6. https://mdn2.netlify.app/en-us/docs/web/svg/element/feblend/
7. https://modern-css.com/motion-path-animation-without-javascript/
8. https://pelicin.netlify.app/web/css/animation/offset
9. https://github.com/mdn/interactive-examples/issues/1643
10. https://www.letsbuildui.dev/articles/what-is-css-motion-path/

11. https://stackoverflow.com/questions/57541032/how-to-apply-the-svg-filter-fegaussianblur-by-avoiding-redundand-svg-elements
12. skills.web_development_stack
13. https://www.perplexity.ai/search/2caf1ea2-14da-4a6b-817f-8685720e7033
14. Screenshot-2026-10-17-at-6.51.38-PM.jpg
15. https://www.w3.org/TR/motion-1/
16. https://www.w3.org/TR/SVG11/filters.html
17. https://developer.mozilla.org/en-US/docs/Web/SVG/Reference/Element/feBlend
18. https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/offset